

SAP
BW4HANA

ZBW Job Monitor

Documentazione Tecnica Completa

Sistema: SAP BW4HANA

Versione ABAP target: 7.5+

Autore: David

Data: Aprile 2026

COMPONENTI PRINCIPALI

ZCL_BW_
JOB_READER

ZCL_TICKET_
DEDUP

ZCL_TICKET_
HTTP_CLIENT

ZCL_TICKET_
FACTORY

ZCL_BW_MAIL_
NOTIFIER

Indice dei Contenuti

- 1. Panoramica del Sistema**
- 2. Architettura**
- 3. Oggetti Dictionary (SE11)**
- 4. Interfacce e Classi ABAP**
- 5. Report e Programmi**
- 6. Configurazione**
 - 6.1 Passo 1 — Destinazione HTTP in SM59
 - 6.2 Passo 2 — Parametri API in ZBWJOB_CONFIG
 - 6.3 Passo 3 — Regole di priorità in ZBWJOB_PRIORITY
 - 6.4 Passo 4 — Mailing list in ZBWJOB_MALLIST
 - 6.5 Requisiti minimi per il primo avvio
- 7. Deploy e Sequenza di Attivazione**
- 8. Schedulazione SM36**
- 9. Gestione Errori**
- 10. Manutenzione e Operatività**
- 11. Estendibilità**
- 12. Glossario Oggetti**
- 13. TODO / Verifiche Pendenti**

1

Panoramica del Sistema

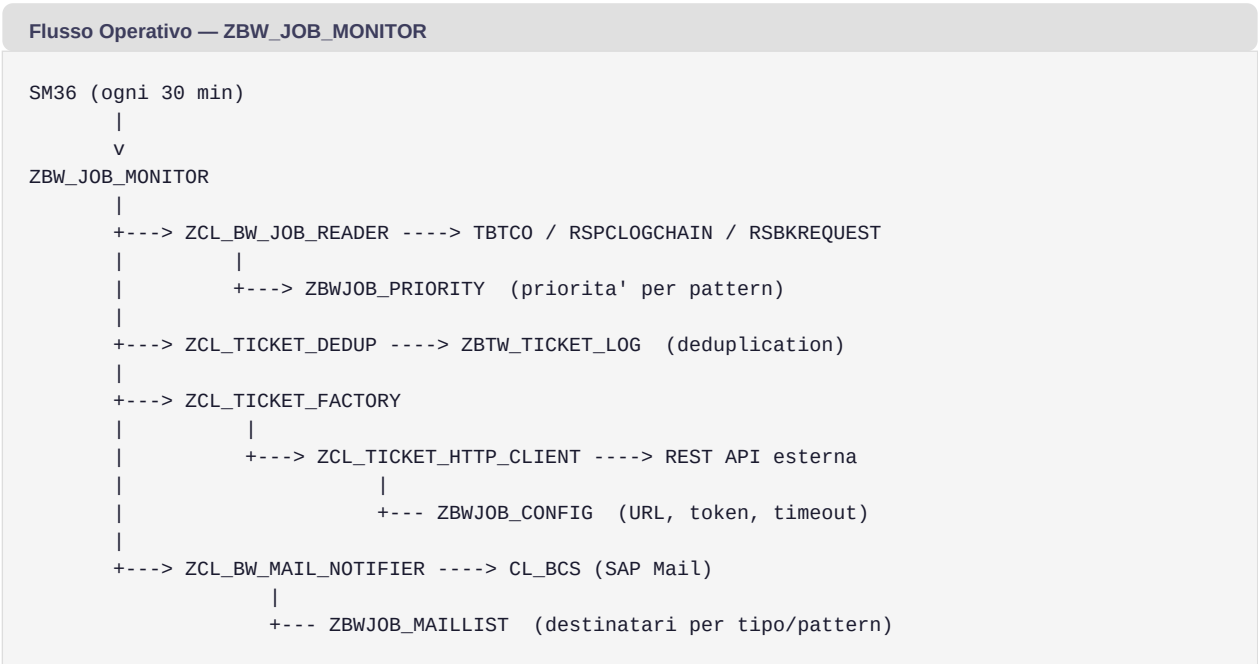
Il sistema **ZBW Job Monitor** monitora automaticamente i job falliti su SAP BW4HANA e crea o aggiorna ticket sul sistema di ticketing aziendale tramite REST API.

Obiettivi principali

Obiettivo	Soluzione
Rilevare job falliti in tempo quasi-reale	Schedulazione ogni 30 min in SM36
Coprire tutti i tipi di oggetti BW	Background Jobs, Process Chains, DTP
Evitare ticket duplicati per lo stesso errore	Tabella di deduplication ZBTW_TICKET_LOG
Assegnare priorità differenziate	Tabella customizing ZBWJOB_PRIORITY
Essere indipendente dal sistema di ticketing	Interfaccia ZIF_TICKET_PROVIDER + factory
Notificare via email i team responsabili all'apertura di un nuovo ticket	Tabella ZBWJOB_MALLIST + classe ZCL_BW_MAIL_NOTIFIER (CL_BCS)
Non abortire il job in caso di errore parziale	Gestione eccezioni per-item con log

Flusso operativo ad alto livello

Il diagramma mostra il flusso end-to-end dall'attivazione SM36 fino alla creazione del ticket e l'invio della notifica email.



2

Architettura

Il sistema è strutturato in quattro layer: Presentazione, Business Logic, Integrazione e Persistenza. Questa separazione consente di sostituire il sistema di ticketing o aggiungere nuove sorgenti dati senza modificare il core.

2.1 Diagramma delle dipendenze

Architettura a Layer — ZBW Job Monitor

LAYER: PRESENTAZIONE

ZBW_JOB_MONITOR (report schedulato)

ZBW_TICKET_STATUS (report consultazione ALV)

| usa

LAYER: BUSINESS LOGIC

ZCL_BW_JOB_READER (lettura job falliti)

ZCL_TICKET_DEDUP (deduplication log)

ZCL_TICKET_FACTORY (creazione provider)

ZCL_BW_MAIL_NOTIFIER (notifiche email)

| implementa

LAYER: INTEGRAZIONE

ZIF_TICKET_PROVIDER (interfaccia)

+-- ZCL_TICKET_HTTP_CLIENT (REST generico)

+-- (futuro) ZCL_JIRA_CLIENT

+-- (futuro) ZCL_SERVICENOW_CLIENT

| legge/scrive

LAYER: PERSISTENZA (SE11)

ZBTW_FAIL_JOB

ZBTW_TICKET_LOG

ZBWJOB_PRIORITY

ZBWJOB_CONFIG

ZBWJOB_MAILLIST

2.2 Pattern architetturali utilizzati

Pattern	Dove applicato	Motivazione
Interface / Strategy	ZIF_TICKET_PROVIDER	Sostituire il sistema di ticketing senza toccare la logica core
Factory Method	ZCL_TICKET_FACTORY	Centralizza la costruzione del provider leggendo la configurazione
Repository	ZCL_TICKET_DEDUP	Isola l'accesso a ZBTW_TICKET_LOG dal resto della logica
Data Reader	ZCL_BW_JOB_READER	Separa la lettura dei dati SAP dalla logica di business
Value Object	ZBTW_FAIL_JOB	Struttura immutabile che trasporta i dati del job tra i layer

3

Oggetti Dictionary (SE11)

3.1 Struttura ZBTW_FAIL_JOB

Tipo: Structure (INTTAB) — non fisica, usata solo come tipo di trasferimento dati tra classi.

Campo	Tipo SAP	Lung.	Descrizione
JOBNAME	BTCJOB	32	Nome del background job (da TBTCO)
JOBCOUNT	BTCJOBCNT	8	Contatore univoco del job run
CHAIN_ID	RSPC_CHAIN	30	ID della Process Chain BW
JOB_TYPE	CHAR10	10	BGDJOB CHAIN DTP
ERROR_MSG	CHAR255	255	Messaggio errore estratto dal log
FAIL_TSTAMP	TIMESTAMP	15	Timestamp UTC del fallimento
PRIORITY	CHAR10	10	HIGH MEDIUM LOW
OBJECT_KEY	CHAR100	100	Chiave univoca: JOBNAME_JOBCOUNT o CHAIN_ID_LOGID

Nota: OBJECT_KEY è la chiave di correlazione usata per deduplication e ticketing.

3.2 Tabella ZBTW_TICKET_LOG

Tipo: Transparent Table — Categoria A (Application Data), Delivery Class A.

Chiave primaria: MANDT + OBJECT_KEY

Campo	Tipo	Ch.	Descrizione
MANDT	MANDT	✓	Client SAP
OBJECT_KEY	CHAR100	✓	Chiave univoca del job/chain
LOG_DATE	DATS		Data di primo rilevamento (non chiave — storico)
TICKET_ID	CHAR50		ID ticket restituito dalla REST API
TICKET_STATUS	CHAR20		OPEN UPDATED CLOSED
CREATED_AT	TIMESTAMP		Timestamp prima apertura ticket
UPDATED_AT	TIMESTAMP		Timestamp ultimo aggiornamento
ERROR_MSG	CHAR255		Ultimo messaggio errore registrato
CHAIN_NAME	CHAR100		Nome leggibile del job/chain

Logica di deduplication: un ticket viene creato una sola volta per OBJECT_KEY, indipendentemente dalla data. Se lo stesso oggetto fallisce più volte (anche a cavallo di mezzanotte), il ticket esistente viene aggiornato con un commento (UPDATED). LOG_DATE registra la data del primo rilevamento ed è usato solo per housekeeping.

Nota: LOG_DATE è stato rimosso dalla chiave primaria per eliminare il rischio di ticket duplicati su fallimenti che attraversano la mezzanotte ("Midnight Duplicate"). La tabella ora ha al massimo una riga per OBJECT_KEY; i re-fallimenti su ticket già CLOSED sovrascrivono la riga esistente tramite MODIFY (vedi ZCL_TICKET_DEDUP).

3.3 Tabella ZBWJOB_PRIORITY

Tipo: Transparent Table — Categoria C (Customizing), Delivery Class G.

Manutenzione: SM30 → vista da creare in SE54.

Chiave primaria: MANDT + CHAIN_PATTERN

Campo	Tipo	Ch.	Descrizione
MANDT	MANDT	✓	Client SAP
CHAIN_PATTERN	CHAR100	✓	Nome esatto o pattern con * (es. ZBW_FIN*)
PRIORITY	CHAR10		HIGH MEDIUM LOW
ACTIVE	XFELD		X = regola attiva
NOTE	CHAR255		Descrizione della regola

Logica di matching in ZCL_BW_JOB_READER→get_priority():

1. Cerca corrispondenza esatta su CHAIN_PATTERN (case-sensitive)
2. Se non trovato, itera le righe con Active = X e usa CP (contains pattern) per il wildcard
3. Default: MEDIUM

Esempio di configurazione:

CHAIN_PATTERN	PRIORITY	NOTE
ZBW_FINANCE_CLOSE	HIGH	Chiusura mese Finance
ZBW_FIN*	HIGH	Tutti i job Finance
ZBW_MASTER*	MEDIUM	Master data loads
ZBW_TEST*	LOW	Ambienti di test

3.4 Tabella ZBWJOB_CONFIG

Tipo: Transparent Table — Categoria C (Customizing), Delivery Class G.

Manutenzione: SM30 → stessa vista di ZBWJOB_PRIORITY o vista separata.

Chiave primaria: MANDT + CONFIG_KEY

Campo	Tipo	Ch.	Descrizione
MANDT	MANDT	✓	Client SAP
CONFIG_KEY	CHAR50	✓	Nome del parametro
CONFIG_VALUE	CHAR255		Valore del parametro
DESCRIPTION	CHAR100		Descrizione uso parametro

Parametri previsti:

CONFIG_KEY	Valore esempio	Descrizione
SM59_DEST	ZBWJOB_TICKETS	Nome della destinazione RFC HTTP creata in SM59 (sostituisce API_URL)
API_PATH	/api/v1/tickets	Path base dell'endpoint REST (separato dall'host gestito in SM59)
API_TOKEN	eyJhbGci...	Bearer token di autenticazione (candidato a migrazione in SECSTORE)
TIMEOUT_SEC	30	Timeout HTTP in secondi
MAX_RETRY	3	Numero massimo di tentativi (informativo)
SYSTEM_ID	BW4PRD	Identificativo sistema nel payload ticket
PACKET_SIZE	20	Numero massimo di job elaborati per run (0 = illimitato). Limita il tempo di esecuzione in caso di mass failure.

3.5 Tabella ZBWJOB_MALLIST

Tipo: Transparent Table — Categoria C (Customizing), Delivery Class G.
Manutenzione: SM30 → vista da creare in SE54.
Chiave primaria: MANDT + JOB_TYPE + CHAIN_PATTERN + EMAIL

Campo	Tipo	Ch.	Descrizione
MANDT	MANDT	✓	Client SAP
JOB_TYPE	CHAR10	✓	Tipo oggetto: BGDJOB CHAIN DTP * (tutti)
CHAIN_PATTERN	CHAR100	✓	Pattern sul nome del job/chain (es. ZBW_FIN*), oppure * per tutti
EMAIL	CHAR241	✓	Indirizzo email del destinatario
ACTIVE	XFELD		X = notifica attiva

Logica di matching in ZCL_BW_MAIL_NOTIFIER→get_recipients():

1. Seleziona tutte le righe con ACTIVE = X e JOB_TYPE IN ('*', <tipo attuale>)
2. Per ogni riga, verifica che il nome del job soddisfi CHAIN_PATTERN tramite CP (contains pattern)
3. Gli indirizzi corrispondenti vengono deduplicati (SORT + DELETE ADJACENT DUPLICATES) prima dell'invio

Nota: Una stessa email può matchare più regole: la deduplication garantisce che ogni destinatario riceva una sola notifica per incident.

Esempi di configurazione:

JOB_TYPE	CHAIN_PATTERN	EMAIL	Effetto
*	*	ops@azienda.com	Notifica ops per qualsiasi tipo e qualsiasi nome
CHAIN	ZBW_FIN*	finance@azienda.com	Solo process chain Finance
DTP	*	dtp-team@azienda.com	Tutti i DTP falliti
BGDJOB	ZBW_HR_LOAD*	hr-it@azienda.com	Solo job HR

Interfacce e Classi ABAP

4.1 Interfaccia ZIF_TICKET_PROVIDER

Scopo: Contratto che ogni implementazione del sistema di ticketing deve rispettare. Permette di sostituire il backend di ticketing senza modificare la logica orchestrante.

File: src/ZIF_TICKET_PROVIDER.intf.abap

Metodo	Direzione	Descrizione
create_ticket(is_job)	→ rv_ticket_id	Apri un nuovo ticket, ritorna l'ID assegnato
update_ticket(iv_ticket_id, iv_message)	—	Aggiunge un commento/update a ticket esistente
get_ticket_status(iv_ticket_id)	→ rv_status	Recupera lo stato corrente del ticket

Tutti i metodi sollevano cx_static_check in caso di errore (HTTP failure, risposta non valida, ecc.).

4.2 Classe ZCL_BW_JOB_READER

Scopo: Legge i job/chain/DTP falliti dalle tabelle SAP standard e li trasforma in oggetti ZBTW_FAIL_JOB.

File: src/ZCL_BW_JOB_READER.clas.abap

Visibilità: PUBLIC FINAL CREATE PUBLIC

Metodi pubblici

Metodo	Parametri	Sorgente dati SAP
get_failed_jobs(iv_lookback_min)	rt_jobs TYPE tt_failed_jobs	TBTCO (STATUS=A) + TBTCP per error msg
get_failed_chains(iv_lookback_min)	rt_jobs TYPE tt_failed_jobs	RSPCLOGCHAIN (LOGSTATE=E/A) + RSPCLOGENTRY
get_failed_dtps(iv_lookback_min)	rt_jobs TYPE tt_failed_jobs	RSBKREQUEST (STATUS=E)

Parametro iv_lookback_min: finestra temporale a ritroso da sy-uzeit corrente. Default 60 minuti. Il timestamp di cutoff viene calcolato da calc_from_timestamp().

Metodi privati

Metodo	Descrizione
build_object_key(iv_name, iv_count)	Concatena NAME_COUNT come chiave univoca
get_priority(iv_chain_name)	Match su ZBWJOB_PRIORITY con fallback MEDIUM

Metodo	Descrizione
calc_from_timestamp(iv_lookback_min)	Restituisce GET TIME STAMP - (min * 60)

Dettaglio sorgenti dati

Background Jobs (TBTCO / TBTCP):

```
TBTCO:  STATUS = 'A' (aborted)
        STRTDATE >= data calcolata da lookback
FOR ALL ENTRIES
TBTCP:  JOBNAME + JOBCOUNT -> campo DYNDTEXT come ERROR_MSG
        (bulk SELECT, poi READ TABLE con SORTED KEY per evitare N+1)
```

Process Chains (RSPCLOGCHAIN / RSPCLOGENTRY):

```
RSPCLOGCHAIN: LOGSTATE IN ('E', 'A')
               STARTTIME >= timestamp lookback
FOR ALL ENTRIES
RSPCLOGENTRY: LOGID + SEVERITY = 'E' -> MESSAGE come ERROR_MSG
               (bulk SELECT in SORTED TABLE con NON-UNIQUE KEY logid)
```

Data Transfer Processes (RSBKREQUEST):

```
RSBKREQUEST: STATUS = '8' (valore assunto - TODO #1: verificare in SE11)
               TIMESTAMP >= timestamp lookback
               GROUP BY dtpname -> un record per DTP, indipendentemente dai pacchetti paralleli
               DTPNAME -> JOBNAME e OBJECT_KEY (senza REQUID per evitare un ticket per pacchetto)
               MSGV1 -> ERROR_MSG
```

4.3 Classe ZCL_TICKET_DEDUP

Scopo: Gestisce la deduplication dei ticket leggendo e scrivendo la tabella ZBTW_TICKET_LOG. Isola completamente la logica di persistenza dal report principale.

File: src/ZCL_TICKET_DEDUP.clas.abap

Visibilità: PUBLIC FINAL CREATE PUBLIC

Metodo	Logica
has_open_ticket(is_job)	SELECT SINGLE su ZBTW_TICKET_LOG per OBJECT_KEY + TICKET_STATUS = 'OPEN' — senza filtro data, per evitare duplicati a cavallo della mezzanotte. Ritorna TICKET_ID se trovato, spazio altrimenti.
save_new_ticket(is_job, iv_ticket_id)	MODIFY su ZBTW_TICKET_LOG con STATUS=OPEN e timestamp corrente. Usa MODIFY (non INSERT) per gestire i re-fallimenti: se il job fallisce di nuovo dopo la chiusura del ticket, la riga CLOSED preesistente viene sovrascritta con il nuovo ticket.
mark_ticket_updated(iv_object_key, iv_message)	UPDATE imposta STATUS=UPDATED, aggiorna ERROR_MSG e UPDATED_AT — senza filtro data.
mark_ticket_closed(iv_object_key)	UPDATE imposta STATUS=CLOSED e aggiorna UPDATED_AT. Chiamato dal report quando il sistema esterno segnala il ticket come chiuso (sync esterno).
cleanup_old_entries(iv_days_to_keep)	DELETE righe con LOG_DATE < sy-datum - iv_days_to_keep

Nota: cleanup_old_entries è chiamato automaticamente alla fine di ogni run del monitor con iv_days_to_keep = 30.

4.4 Classe ZCL_TICKET_HTTP_CLIENT

Scopo: Implementazione concreta di ZIF_TICKET_PROVIDER che comunica con il sistema di ticketing tramite chiamate HTTP REST generiche.

File: src/ZCL_TICKET_HTTP_CLIENT.clas.abap

Visibilità: PUBLIC FINAL CREATE PUBLIC

Implementa: ZIF_TICKET_PROVIDER

Constructor

ABAP Constructor

```
NEW zcl_ticket_http_client(  
    iv_destination = 'ZBWJOB_TICKETS'    " SM59 RFC destination name  
    iv_api_path    = '/api/v1/tickets'    " base path, set via ~request_uri  
    iv_api_token   = 'eyJ...'              
    iv_timeout     = 30 ).
```

Payload JSON inviato su create_ticket

JSON — create_ticket

```
{
  "title": "BW Job Failed: <JOBNAME>",
  "description": "<ERROR_MSG>",
  "priority": "<HIGH|MEDIUM|LOW>",
  "system_id": "<SYSTEM_ID da ZBWJOB_CONFIG>",
  "object_key": "<OBJECT_KEY>",
  "job_type": "<BGDJOB|CHAIN|DTP>",
  "failed_at": "<FAIL_TSTAMP>"
}
```

Payload JSON inviato su update_ticket

JSON — update_ticket

```
{
  "comment": "<ERROR_MSG>",
  "status": "OPEN"
}
```

Parsing del ticket_id dalla risposta

Il metodo parse_ticket_id() cerca in sequenza:

- 1. "id": "string"
- 2. "ticket_id": "string"
- 3. "id": 12345 (numerico)

Gestione HTTP

Aspetto	Implementazione
Client HTTP	cl_http_client=>create_by_destination() — host, porta e SSL gestiti in SM59
Path endpoint	Header ~request_uri impostato su mv_api_path (o mv_api_path/ticket_id per update/get)
Header Auth	Authorization: Bearer <token>
Content-Type	application/json
Popup logon	co_disabled — disabilitato per esecuzione background
Timeout	Passato a send(EXPORTING timeout = mv_timeout)
Codici successo	200 e 201; qualsiasi altro codice → RAISE cx_static_check
Serializzazione JSON	/ui2/cl_json=>serialize() su strutture tipizzate (ty_create_payload, ty_update_payload). Garantisce escape corretto di ", \, newline, tab e tutti i caratteri di controllo.

4.5 Classe ZCL_TICKET_FACTORY

Scopo: Factory che legge ZBWJOB_CONFIG e istanzia il provider corretto. Punto di estensione per future implementazioni multi-provider.

File: src/ZCL_TICKET_FACTORY.clas.abap

Visibilità: PUBLIC FINAL CREATE PRIVATE (Singleton-like, solo CLASS-METHODS)

Utilizzo

```
DATA(lo_provider) = zcl_ticket_factory=>get_provider( ).
```

Logica interna di get_provider():

1. Legge ZBWJOB_CONFIG per SM59_DEST, API_PATH, API_TOKEN, TIMEOUT_SEC
2. Se TIMEOUT_SEC è vuoto o zero → default 30 secondi
3. Istanza e ritorna ZCL_TICKET_HTTP_CLIENT con destinazione SM59 e path

Attenzione: Sicurezza: API_TOKEN è ancora letto da ZBWJOB_CONFIG. Per ambienti produttivi, migrarlo in SECSTORE / CL_SECURE_STORE_SMC e aggiornare la factory di conseguenza.

Nota: Estensione futura: aggiungere CONFIG_KEY = 'PROVIDER_TYPE' (es. JIRA, SERVICENOW) e un CASE nella factory per scegliere l'implementazione.

4.6 Classe ZCL_BW_MAIL_NOTIFIER

Scopo: Invia una email di notifica ai destinatari configurati in ZBWJOB_MALLIST ogni volta che viene aperto un **nuovo** ticket. Non viene invocata su UPDATE o CLOSE.

File: src/ZCL_BW_MAIL_NOTIFIER.clas.abap

Visibilità: PUBLIC FINAL CREATE PUBLIC

Dipendenza SAP: CL_BCS / CL_DOCUMENT_BCS / CL_INTERNET_ADDRESS_BCS (ABAP Messaging Services)

Metodo pubblico

Metodo	Parametri	Descrizione
notify_new_ticket(is_job, iv_ticket_id)	—	Entry point: recupera destinatari, costruisce body, invia tramite CL_BCS

Metodi privati

Metodo	Descrizione
get_recipients(is_job)	SELECT su ZBWJOB_MALLIST, filtraggio per job type e CP pattern, dedup indirizzi
get_job_name(is_job)	Restituisce chain_id se presente, altrimenti jobname, altrimenti object_key

Metodo	Descrizione
<code>build_mail_body(is_job, iv_ticket_id, iv_job_name)</code>	Costruisce il corpo testuale (<code>bcsy_text</code>) con i campi dell'incident

Esempio corpo email generato

Esempio Email

```
BW4HANA Job Monitor - Incident Notification
-----

Ticket ID   : TKT-00123
Job Name    : ZBW_FINANCE_CLOSE
Job Type    : CHAIN
Priority     : HIGH
Failed at   : 20260413140000

Error Message:
Step LOAD_GL_DATA ended with error: database connection lost

Object Key  : ZBW_FINANCE_CLOSE_000000001234

This is an automated message from ZBW_JOB_MONITOR.
```

Gestione errori

Gli errori di invio email (`CX_BCS`) vengono catturati internamente e loggati con `WRITE: [MAIL-ERR]`. L'eccezione non viene mai propagata al report principale: un problema SMTP non deve bloccare la creazione del ticket.

5

Report e Programmi

5.1 Report ZBW_JOB_MONITOR

File: src/ZBW_JOB_MONITOR.prog.abap
Tipo: ABAP Report (schedulato in SM36)

Parametri di selezione

Parametro	Tipo	Default	Descrizione
P_LKBK	I	60	Finestra di lookback in minuti
P_TEST	XFELD	(spazio)	Modalità test: nessun ticket viene aperto/aggiornato

Flusso di esecuzione

Flusso di Esecuzione — ZBW_JOB_MONITOR

```

START-OF-SELECTION
|
| +- 1. ZCL_BW_JOB_READER:
|   |   get_failed_jobs(P_LKBK)    -> TBTC0
|   |   get_failed_chains(P_LKBK)  -> RSPCLOGCHAIN
|   |   get_failed_dtps(P_LKBK)    -> RSBKREQUEST
|   |   +- APPEND LINES -> LT_ALL_JOBS
|   |
| +- 2. Se LT_ALL_JOBS e' vuota -> WRITE messaggio -> RETURN
|   |
| +- 3. Istanzia ZCL_TICKET_DEDUP, ZCL_TICKET_FACTORY=>get_provider()
|   |   e ZCL_BW_MAIL_NOTIFIER
|   |   Legge PACKET_SIZE da ZBWJOB_CONFIG (0 = illimitato)
|   |
| +- 4. LOOP AT LT_ALL_JOBS:
|   |   |
|   |   +- [PACKET_SIZE] Se job elaborati > PACKET_SIZE -> WRITE [LIMIT] -> EXIT
|   |   |
|   |   +- has_open_ticket() -> ticket aperto in SAP?
|   |   |
|   |   |   +- SI: [se P_TEST=' ']
|   |   |   |   |   get_ticket_status() -> stato nel sistema esterno?
|   |   |   |   |   +- CLOSED: mark_ticket_closed() -> COMMIT WORK AND WAIT
|   |   |   |   |   |   WRITE: [SYNC] ticket chiuso esternamente
|   |   |   |   |   +- OPEN:  update_ticket() + mark_ticket_updated()
|   |   |   |   |   |   COMMIT WORK AND WAIT
|   |   |   |   |   |   WRITE: [UPDATE] ticket_id - object_key
|   |   |   |   |   |   [se P_TEST='X'] WRITE: [UPDATE] (simulato)
|   |   |   |   |
|   |   |   |   +- NO: create_ticket() + save_new_ticket() [se P_TEST=' ']
|   |   |   |   |   COMMIT WORK AND WAIT
|   |   |   |   |   notify_new_ticket(is_job, lv_new_id) <- ZBWJOB_MAILLIST
|   |   |   |   |   WRITE: [CREATE] ticket_id - object_key (prio)
|   |   |   |
|   |   |   +- CATCH cx_static_check -> WRITE [ERROR] + MESSAGE 'I'
|   |   |
|   |
| +- 5. cleanup_old_entries(30)
|   |   WRITE: 'Housekeeping complete.'

```

Modalità TEST

Con P_TEST = X:

- Tutti i SELECT e la logica di rilevamento girano normalmente
- Nessuna chiamata HTTP viene effettuata
- Nessun record viene scritto in ZBTW_TICKET_LOG
- L'output mostra [TEST - not created] al posto del ticket ID reale

Utile per validare la lettura dei log SAP senza effetti collaterali.

5.2 Report ZBW_TICKET_STATUS

File: src/ZBW_TICKET_STATUS.prog.abap

Tipo: ABAP Report interattivo (ALV)

Parametri di selezione

Parametro	Tipo	Default	Descrizione
P_DAYFR	DATS	sy-datum	Data inizio filtro
P_DAYTO	DATS	sy-datum	Data fine filtro
S_STATUS	Select-option	OPEN	Filtro su TICKET_STATUS

Funzionalità ALV

- Display ottimizzato con tutte le colonne di ZBTW_TICKET_LOG
- Toolbar standard attiva (export Excel/PDF, filtri, sort)
- **Pulsante custom "Close Ticket"** nella toolbar destra:
 1. Recupera le righe selezionate via `cl_salv_selections`
 2. Esegue `UPDATE ZBTW_TICKET_LOG SET TICKET_STATUS = 'CLOSED'` per ogni riga (WHERE `OBJECT_KEY` — non usa più `LOG_DATE` che non è più chiave)
 3. Aggiorna `UPDATED_AT` con il timestamp corrente
 4. Emette `COMMIT WORK` per persistere tutte le modifiche prima del refresh
 5. Chiama `go_alv->refresh()` per riflettere i cambiamenti in-place
 6. Mostra un messaggio 'N ticket(s) closed.'

Configurazione

Ci sono tre aree da configurare prima di avviare il monitor. Seguire l'ordine indicato: prima SM59, poi ZBWJOB_CONFIG, poi ZBWJOB_PRIORITY.

6.1 Passo 1 — Creare la destinazione HTTP in SM59

SM59 gestisce host, porta e certificati SSL in modo centralizzato e sicuro, evitando di memorizzare URL completi in tabelle trasportabili.

Cosa ti serve prima di iniziare:

- URL base del tuo sistema di ticketing (es. <https://tickets.azienda.com>)
- Eventuale certificato SSL da importare in STRUST (richiede supporto Basis)

Step 1.1 — Aprire SM59 e creare la destinazione

1. Aprire la transazione **SM59**
2. Cliccare il pulsante **Crea** (o premere F5)
3. Nel campo *Destination*, inserire un nome identificativo — es. ZBWJOB_TICKETS (annota questo nome: servirà dopo)
4. Nel campo *Connection Type*, selezionare **G — HTTP connection to external server**
5. Cliccare **Invio** per aprire il form di dettaglio

Step 1.2 — Tab "Technical Settings"

Campo	Valore da inserire
Target Host	Solo il dominio, senza https:// — es. tickets.azienda.com
Service No.	443 per HTTPS (raccomandato), 80 per HTTP
Path Prefix	Lasciare vuoto — il path viene gestito da API_PATH in ZBWJOB_CONFIG

Step 1.3 — Tab "Logon & Security"

Campo	Valore da inserire
SSL	Impostare su Active
SSL Client Certificate	Selezionare il certificato corrispondente al server di destinazione (importato in STRUST)
Logon Procedure	Nessuna — l'autenticazione avviene via Bearer token nell'header HTTP

Step 1.4 — Testare la connessione

1. Cliccare il pulsante **Connection Test** in SM59
2. Verificare che la risposta sia 200 OK o 401 Unauthorized (entrambi confermano la raggiungibilità di rete — il 401 è atteso senza token)
3. Se il test fallisce con errore di rete o SSL, coinvolgere il team Basis prima di proseguire

Step 1.5 — Salvare

1. Cliccare **Salva** (Ctrl+S)

6.2 Passo 2 — Inserire i parametri API in ZBWJOB_CONFIG

Cosa ti serve prima di iniziare:

- Nome della destinazione SM59 creata al Passo 1
- URL completo dell'endpoint API del tuo sistema di ticketing (dalla sua documentazione)
- Un Bearer token valido con permessi di creazione e aggiornamento ticket

Step 2.1 — Aprire la vista di manutenzione

1. Aprire la transazione **SM30**
2. Nel campo *Table/View*, inserire il nome della vista di manutenzione di ZBWJOB_CONFIG
3. Cliccare **Maintain** (F5)

Step 2.2 — Inserire i parametri obbligatori

Creare un record per ciascuna delle seguenti chiavi cliccando **Nuova voce**:

CONFIG_KEY	Cosa inserire	Dove trovarlo
SM59_DEST	Nome della destinazione SM59 — es. ZBWJOB_TICKETS	Creata al Passo 1
API_PATH	Path base dell'endpoint — es. /api/v1/tickets	Documentazione API del sistema di ticketing
API_TOKEN	Bearer token — es. eyJhbGciOiJIUzI1NiIs...	Generato nel pannello API keys del sistema di ticketing
SYSTEM_ID	Etichetta identificativa di questo sistema SAP — es. BW4PRD	Libero, comparirà nel corpo di ogni ticket

Attenzione: API_TOKEN: il token deve avere i permessi di creazione (POST) e aggiornamento (PUT/POST) ticket. Verificare nella documentazione del sistema di ticketing come generarlo. In ambienti produttivi valutare la migrazione in SECSTORE per evitare esposizione in SM30.

Step 2.3 — Inserire i parametri opzionali

CONFIG_KEY	Valore consigliato	Descrizione
TIMEOUT_SEC	30	Timeout in secondi per ogni chiamata HTTP. Aumentare se la rete è lenta o il sistema di ticketing è lento a rispondere
PACKET_SIZE	20	Numero massimo di job elaborati per run. Utile per limitare il tempo di esecuzione in caso di mass failure. 0 = nessun limite
MAX_RETRY	3	Informativo — non ancora enforced nel codice

Step 2.4 — Salvare

1. Cliccare **Salva** (Ctrl+S)
2. Assegnare a un ordine di trasporto se richiesto

Risultato atteso in tabella:

ZBWJOB_CONFIG — Risultato atteso		
CONFIG_KEY	CONFIG_VALUE	DESCRIPTION
SM59_DEST	ZBWJOB_TICKETS	SM59 RFC destination name
API_PATH	/api/v1/tickets	Base path REST endpoint
API_TOKEN	eyJhbGciOiJIUzI1NiIs...	Bearer auth token
SYSTEM_ID	BW4PRD	System ID in ticket payload
TIMEOUT_SEC	30	HTTP timeout (seconds)
PACKET_SIZE	20	Max jobs per run (0 = no limit)
MAX_RETRY	3	Max retry attempts

6.3 Passo 3 — Configurare le regole di priorità in ZBWJOB_PRIORITY

Cosa ti serve prima di iniziare:

- Elenco dei nomi dei job/chain/DTP critici nel sistema BW (da SM37 o RSA1)

Nota: Questo passo è **opzionale** per un primo avvio. Se la tabella è vuota, tutti i job riceveranno priorità MEDIUM come default.

Step 3.1 — Aprire la vista di manutenzione

1. Aprire la transazione **SM30**
2. Inserire il nome della vista di manutenzione di ZBWJOB_PRIORITY
3. Cliccare **Maintain**

Step 3.2 — Inserire le regole

Cliccare **Nuova voce** per ogni regola. Compilare i campi:

Campo	Cosa inserire
CHAIN_PATTERN	Nome esatto del job/chain oppure pattern con * (es. ZBW_FIN* copre tutti i job che iniziano con ZBW_FIN)
PRIORITY	HIGH, MEDIUM oppure LOW
ACTIVE	Mettere X per attivare la regola
NOTE	Descrizione libera — utile per documentare perché quella priorità

Nota: Logica di matching: il sistema cerca prima una corrispondenza esatta sul nome del job. Se non trovata, scorre le regole con wildcard nell'ordine in cui appaiono in tabella e si ferma alla prima corrispondenza. Inserire sempre le regole **dalla più specifica alla più generica**.

Esempio di configurazione:

ZBWJOB_PRIORITY — Esempio

CHAIN_PATTERN	PRIORITY	ACTIVE	NOTE
ZBW_FINANCE_CLOSE_EOD	HIGH	X	Chiusura giornata Finance
ZBW_FIN*	HIGH	X	Tutti i job Finance
ZBW_HR*	HIGH	X	HR critical loads
ZBW_MASTER_DATA*	MEDIUM	X	Master data load
ZBW_DELTA*	MEDIUM	X	Delta loads standard
ZBW_TEST*	LOW	X	Job di test/sviluppo

Step 3.3 — Salvare

1. Cliccare **Salva** (Ctrl+S)

6.4 Passo 4 — Configurare la mailing list in ZBWJOB_MALLIST

Nota: Questo passo è **opzionale**. Se la tabella è vuota, nessuna email viene inviata ma il monitor funziona normalmente.

Cosa ti serve prima di iniziare:

- Indirizzi email dei team da notificare
- SAP configurato per l'invio email (transazione SCOT — da verificare con il team Basis)

Step 4.1 — Aprire la vista di manutenzione

1. Aprire la transazione **SM30**
2. Inserire il nome della vista di manutenzione di ZBWJOB_MALLIST
3. Cliccare **Maintain**

Step 4.2 — Inserire le regole di notifica

Cliccare **Nuova voce** per ogni combinazione tipo/pattern/email:

Campo	Cosa inserire
JOB_TYPE	BGDJOB, CHAIN, DTP oppure * per qualsiasi tipo
CHAIN_PATTERN	Nome esatto o pattern con * (stesso formato di ZBWJOB_PRIORITY)
EMAIL	Indirizzo email del destinatario (max 241 caratteri)
ACTIVE	X per abilitare la riga

Nota: La stessa email può apparire in più righe (es. un indirizzo generico ops@ e uno specifico per Finance): la classe ZCL_BW_MAIL_NOTIFIER deduplica automaticamente i destinatari prima dell'invio.

Esempio di configurazione:

ZBWJOB_MALLIST — Esempio			
JOB_TYPE	CHAIN_PATTERN	EMAIL	ACTIVE
*	*	ops-bw@azienda.com	X
CHAIN	ZBW_FIN*	finance-it@azienda.com	X
DTP	*	dtp-monitoring@azienda.com	X
BGDJOB	ZBW_HR*	hr-it@azienda.com	X

Step 4.3 — Salvare

1. Cliccare **Salva** (Ctrl+S)

6.5 Requisiti minimi per il primo avvio

Se vuoi avviare il monitor il prima possibile, i passi strettamente necessari sono:

#	Cosa fare	Dove
1	Creare destinazione SM59 verso il sistema di ticketing	SM59
2	Inserire SM59_DEST, API_PATH, API_TOKEN, SYSTEM_ID	SM30 → ZBWJOB_CONFIG
3	Eseguire ZBW_JOB_MONITOR con P_TEST = X	SE38

Il test in modalità P_TEST = X mostra i job falliti rilevati senza aprire alcun ticket reale, permettendo di verificare configurazione e connettività prima della messa in produzione.

7

Deploy e Sequenza di Attivazione

Seguire rigorosamente questa sequenza per evitare errori di dipendenza:

Step 1 — DDIC (SE11)

1. Aprire SE11
2. Creare e attivare ZBTW_FAIL_JOB (Structure) — importare da src/ddic/ZBTW_FAIL_JOB.xml via abapGit o creare manualmente
3. Creare e attivare ZBTW_TICKET_LOG (Tabella)
4. Creare e attivare ZBWJOB_PRIORITY (Tabella)
5. Creare e attivare ZBWJOB_CONFIG (Tabella)
6. Creare e attivare ZBWJOB_MAILLIST (Tabella)
7. Eseguire conversione dati (SE14) se necessario

Step 2 — Destinazione SM59

1. SM59 → Creare destinazione RFC di tipo G (HTTP connection to external server) con il nome che sarà inserito in SM59_DEST
2. Configurare host, porta, SSL e certificato nel tab "Logon & Security"
3. Eseguire il connection test integrato in SM59 per validare raggiungibilità

Step 3 — Dati di configurazione

1. SM30 → Vista ZBWJOB_CONFIG: inserire almeno SM59_DEST, API_PATH, API_TOKEN, SYSTEM_ID
2. SM30 → Vista ZBWJOB_PRIORITY: inserire le regole di priorità base

Step 4 — Interfaccia (SE24)

1. Creare ZIF_TICKET_PROVIDER con i tre metodi definiti

Step 5 — Classi (SE24), in ordine

1. ZCL_BW_JOB_READER — nessuna dipendenza su altre classi Z
2. ZCL_TICKET_DEDUP — dipende da ZBTW_TICKET_LOG
3. ZCL_TICKET_HTTP_CLIENT — implementa ZIF_TICKET_PROVIDER
4. ZCL_TICKET_FACTORY — dipende da ZCL_TICKET_HTTP_CLIENT e ZBWJOB_CONFIG
5. ZCL_BW_MAIL_NOTIFIER — dipende da ZBWJOB_MAILLIST e CL_BCS

Step 6 — Report (SE38)

1. ZBW_JOB_MONITOR — dipende da tutte le classi precedenti
2. ZBW_TICKET_STATUS — dipende da ZBTW_TICKET_LOG

Step 7 — Test isolato

```
SE38 -> ZBW_JOB_MONITOR -> Eseguire con P_TEST = X
```

Verificare che l'output mostri i job falliti senza errori di sintassi o runtime.

Step 8 — Test integrazione (facoltativo)

Configurare la destinazione SM59 puntando a un mock server (es. Postman Mock, RequestBin) e rieseguire con P_TEST = '' per validare il payload HTTP e il parsing del ticket ID.

Step	Transazione	Oggetti	Dipendenze
1	SE11	ZBTW_FAIL_JOB, ZBTW_TICKET_LOG, ZBWJOB_*	Nessuna
2	SM59	Destinazione HTTP tipo G	Certificato SSL in STRUST
3	SM30	ZBWJOB_CONFIG, ZBWJOB_PRIORITY	Step 1
4	SE24	ZIF_TICKET_PROVIDER (interfaccia)	Step 1
5	SE24	ZCL_BW_JOB_READER, ZCL_TICKET_DEDUP, ZCL_TICKET_HTTP_CLIENT, ZCL_TICKET_FACTORY, ZCL_BW_MAIL_NOTIFIER	Step 4
6	SE38	ZBW_JOB_MONITOR, ZBW_TICKET_STATUS	Step 5
7	SE38	Test con P_TEST=X	Step 2–6

Schedulazione SM36

Variante PROD (produzione)

SM36 — Variante PROD

Job Name: ZBW_JOB_MONITOR_PROD

Step 1: ABAP Program = ZBW_JOB_MONITOR

Variante: PROD

 P_LKBK = 60

 P_TEST = (spazio)

Schedule: Periodico -- ogni 30 minuti

 Start time: immediatamente dopo l'attivazione

Nota: Con lookback 60 minuti e run ogni 30 minuti c'è una sovrapposizione intenzionale: garantisce che nessun job fallito venga saltato in caso di ritardo del ciclo precedente. La deduplication evita ticket doppi.

Variante TEST

SM36 — Variante TEST

Job Name: ZBW_JOB_MONITOR_TEST

Step 1: ABAP Program = ZBW_JOB_MONITOR

Variante: TEST

 P_LKBK = 60

 P_TEST = X

Schedule: Manuale (on-demand)

Autorizzazioni richieste

Oggetto autorizzazione	Campo	Valore
S_BTCH_JOB	JOBACTION	RELE
S_RFC	FUGR	(per chiamate HTTP)
S_TABU_DIS	ACTVT	02, 03 su gruppo tabella Z

Gestione Errori

Filosofia

Il report ZBW_JOB_MONITOR **non deve abortire** per un errore su un singolo job. La gestione è a due livelli:

Struttura Try/Catch a due livelli

```
TRY (esterno -- errori fatali: configurazione mancante, HTTP setup)
|
LOOP AT lt_all_jobs
  TRY (per-job -- errori su singolo ticket)
    CATCH cx_static_check
      -> WRITE [ERROR] + MESSAGE 'I' <-- non interrompe il loop
    ENDTRY
  ENDLLOOP
|
  cleanup_old_entries()
|
CATCH cx_static_check <-- errori fatali
  -> WRITE [FATAL] + MESSAGE 'I'
ENDTRY
```

Tipi di errore e comportamento

Scenario	Comportamento
Job SAP senza error message in TBTCP	ERROR_MSG resta vuoto; ticket aperto senza descrizione
API non raggiungibile (timeout)	cx_static_check catturata per-job; log su WRITE; job continua
HTTP status code != 200/201	Come sopra
parse_ticket_id non trova ID	rv_ticket_id = spazio; save_new_ticket persiste record con ticket ID vuoto
ZBWJOB_CONFIG mancante	ZCL_TICKET_FACTORY usa fallback per timeout; URL/token restano vuoti → errore HTTP
Ticket chiuso esternamente, SAP ancora OPEN	get_ticket_status() rileva lo stato CLOSED; il report chiama mark_ticket_closed() e logga [SYNC]. Il fallimento della sola chiamata di status check (eccezione) è gestito silenziosamente: si procede con l'update standard.
PACKET_SIZE raggiunto	Il loop termina con [LIMIT]; i job rimanenti vengono elaborati al prossimo run del monitor.
Job fallisce di nuovo dopo chiusura ticket	has_open_ticket() non trova ticket OPEN; save_new_ticket() sovrascrive la riga CLOSED tramite MODIFY e apre un nuovo ticket.

Nota: Controllare sy-subrc dopo MODIFY in ZCL_TICKET_DEDUP→save_new_ticket() è un miglioramento raccomandato per ambienti produttivi.

Log applicativo (opzionale — produzione)

Per un log persistente consultabile via SLG1, sostituire i WRITE con:

Esempio APPL_LOG

```
CALL FUNCTION 'APPL_LOG_WRITE_MESSAGE'
  EXPORTING
    object      = 'ZBWMONITOR'
    subobject   = 'TICKETS'
    ...
```

Manutenzione e Operatività

Consultazione ticket aperti

1. Eseguire ZBW_TICKET_STATUS in SE38 o tramite transazione dedicata
2. Filtrare per data e status desiderato
3. Selezionare le righe e cliccare **"Close Ticket"** per chiuderle manualmente

Housekeeping automatico

Il report ZBW_JOB_MONITOR chiama automaticamente cleanup_old_entries(30) al termine di ogni run. Modifica il parametro direttamente nel sorgente se serve una retention diversa.

Pulizia manuale

Per svuotare la log manualmente (es. in sistemi di test):

SQL — Pulizia manuale

```
-- SM30 o SE16N  
DELETE FROM zbtw_ticket_log WHERE log_date < '20260101'.
```

Monitoraggio del job stesso

Il job ZBW_JOB_MONITOR_PROD deve essere monitorato tramite SM37. In caso di abend, verificare:

1. Configurazione ZBWJOB_CONFIG (token scaduto, URL cambiato)
2. Raggiungibilità rete del sistema di ticketing dall'application server SAP
3. Autorizzazioni ABAP per le SELECT sulle tabelle standard (TBTCO, RSPCLOGCHAIN, ecc.)

Estendibilità

Aggiungere un nuovo sistema di ticketing (es. Jira)

1. Creare una nuova classe ZCL_JIRA_CLIENT che implementa ZIF_TICKET_PROVIDER
2. Implementare i tre metodi (create_ticket, update_ticket, get_ticket_status) con le API Jira
3. In ZBWJOB_CONFIG, aggiungere CONFIG_KEY = 'PROVIDER_TYPE' con valore JIRA
4. In ZCL_TICKET_FACTORY→get_provider(), aggiungere:

ABAP — Estensione Factory per Jira

```
CASE lv_provider_type.  
  WHEN 'JIRA'.  
    ro_provider = NEW zcl_jira_client( ... ).  
  WHEN OTHERS.  
    ro_provider = NEW zcl_ticket_http_client( ... ).  
ENDCASE.
```

Nessuna modifica al report ZBW_JOB_MONITOR o alle altre classi.

Aggiungere un nuovo tipo di oggetto BW monitorato

1. Aggiungere un metodo get_failed_<tipo>() in ZCL_BW_JOB_READER
2. Implementare la logica di SELECT sulla tabella SAP corrispondente
3. In ZBW_JOB_MONITOR, aggiungere APPEND LINES OF lo_reader->get_failed_<tipo>(p_lkbk) TO lt_all_jobs

Modificare o estendere le regole di notifica email

Le regole di destinazione sono in ZBWJOB_MALLIST e vengono lette a runtime: nessuna modifica al codice è necessaria per aggiungere o rimuovere destinatari. Per aggiungere un nuovo criterio di filtraggio (es. notifica solo per PRIORITY = 'HIGH'), estendere ZCL_BW_MAIL_NOTIFIER→get_recipients() o notify_new_ticket() con la logica aggiuntiva.

Nota: Prerequisito infrastrutturale: il server ABAP deve essere configurato per l'invio email tramite SCOT (transazione SAP). Coinvolgere il team Basis prima di attivare le notifiche in produzione.

Glossario Oggetti

Oggetto SAP	Tipo	Descrizione
ZBTW_FAIL_JOB	Structure (SE11)	Struttura dati per job fallito
ZBTW_TICKET_LOG	Transparent Table	Log deduplication ticket
ZBWJOB_PRIORITY	Transparent Table	Regole di priorità per pattern
ZBWJOB_CONFIG	Transparent Table	Parametri connessione API
ZBWJOB_MAILLIST	Transparent Table	Regole mailing list per tipo/pattern
ZIF_TICKET_PROVIDER	Interface	Contratto provider ticketing
ZCL_BW_JOB_READER	Class	Lettore job falliti da SAP standard
ZCL_TICKET_DEDUP	Class	Gestore deduplication ticket
ZCL_TICKET_HTTP_CLIENT	Class	Implementazione REST generica
ZCL_TICKET_FACTORY	Class	Factory per il provider
ZCL_BW_MAIL_NOTIFIER	Class	Invio email notifiche via CL_BCS
ZBW_JOB_MONITOR	Report	Orchestratore principale (SM36)
ZBW_TICKET_STATUS	Report	Consultazione ALV + close manuale

13

TODO / Verifiche Pendenti

Questa sezione raccoglie le verifiche aperte e le attività da completare prima della messa in produzione.

#	Priorità	Stato	Componente	Descrizione	Come verificare
1	ALTA	Aperto	ZCL_BW_JOB_READER	Verificare il valore esatto del campo status in RSBKREQUEST per i DTP falliti. Il codice attuale usa status = '8' (assunto durante il test logico sul campione dati), ma non è stato confermato tramite SE11. Se il valore reale è diverso, i DTP falliti non vengono mai rilevati.	SE11 → RSBKREQUEST → campo "Stato di elaborazione tecnico" → dominio → valori fissi. Oppure: RSMO → trovare un DTP fallito noto → annotare il REQUID → SE16N → verificare il valore del campo status per quel record.
2	ALTA	Aperto	ZCL_BW_JOB_READER	Verificare il nome tecnico esatto del campo status in RSBKREQUEST. Il codice usa il campo generico status ma il nome reale potrebbe essere diverso (es. RSTSTATUS).	SE11 → RSBKREQUEST → lista campi → cercare il campo corrispondente a "Stato di elaborazione tecnico" e confrontare con il nome usato nel WHERE.
3	MEDIA	Risolto	ZCL_BW_JOB_READER	Un ticket per DTP, non per pacchetto parallelo. In presenza di esecuzioni DTP parallele, più REQUID falliti per lo stesso DTPNAME generavano un ticket per ogni pacchetto. Risolto con GROUP BY dtpname nella SELECT e usando solo il nome DTP come OBJECT_KEY (senza REQUID).	—
4	MEDIA	Aperto	ZCL_BW_JOB_READER	DYNDTEXT assente nel campione TBTCP. Nel test logico il campo DYNDTEXT (testo errore del passo job) non era visibile nell'export CSV. Verificare che sia effettivamente popolato in produzione per i job abortiti, altrimenti i ticket vengono aperti con description vuota.	SE16N → TBTCP → filtrare per un job abortito noto → verificare se il campo DYNDTEXT contiene testo.
5	BASSA	Aperto	API_TOKEN	Migrare il Bearer token da ZBWJOB_CONFIG a SECSTORE. Attualmente il token è in chiaro in una tabella trasportabile e visibile in SM30.	Usare CL_SECURE_STORE_SMC per lettura/scrittura sicura e aggiornare ZCL_TICKET_FACTORY → get_provider().

Documentazione generata il 13 aprile 2026.